RA2: Django + React con Autenticación JWT y APIs

Desarrollo Full Stack con Django REST Framework, React (Vite), SOAP y API de Terceros
Manual de Desarrollo: Páginas 13 a 35

1. Información del Proyecto y Repositorio en GitHub

El presente trabajo corresponde al cierre de las Unidades 2 y 3. Se tomó como base el Backend desarrollado en Django con autenticación JWT y se construyó un Frontend completo en React utilizando Vite, implementando control de rutas, layouts, persistencia de sesión con Context API y validación de formularios con Formik y Yup.

Enlace al Repositorio: https://github.com/ElMandinho777/django

Rama principal: practica-usuarios-jwt / master (incluye carpetas backend y frontend, README.md explicativo, requirements.txt y .gitignore que excluye el entorno virtual venv/ y la base de datos db.sqlite3).

2. Estructura y Arquitectura Desarrollada

El proyecto se organizó en dos módulos independientes bajo una misma solución:

• **Backend (Django):** API desarrollada con Django 5 y Django REST Framework. Incluye SimpleJWT para la emisión y renovación de tokens de acceso (30 minutos) y refresco (24 horas), serializadores con validación de seguridad (contraseñas en write_only con hashing PBKDF2), configuración de CORS (django-cors-headers), endpoints de usuario (/api/v1/users/), perfil personal (/api/v1/auth/me/), registro público (/api/v1/auth/register/) y un Web Service SOAP.

• **Frontend (React):** Aplicación creada con Vite, Semantic UI React y SASS. Implementa React Router v6 con tres esquemas de layouts (AdminLayout, ClientLayout, CommonLayout), enrutamiento dinámico, AuthContext para conservar el estado global de la sesión mediante Custom Hooks (useAuth, useUser), almacenamiento del token en localStorage y consumo asíncrono con fetch.

• **API de Terceros:** Integración de un widget en el Dashboard que consume una API pública gratuita (Quotes API) para renderizar contenido externo dinámico.

• **Web Service SOAP:** Endpoint propio en Django que utiliza la librería Zeep para consumir el servicio público WSDL de CountryInfoService, devolviendo información normalizada del país en formato JSON.

3. Evidencias de Ejecución (Capturas de Pantalla)

3.1 Login Funcionando con Obtención de Token JWT

En la pantalla de Login de React (/login), se ingresaron las credenciales del usuario (correo y contraseña). Al enviar el formulario validado con Formik y Yup, la aplicación realiza una petición POST hacia /api/v1/auth/token/ en Django. Como se observa en la consola de desarrollo, el backend responde con código HTTP 200 y entrega el token de acceso JWT, el cual es procesado y almacenado de forma segura.

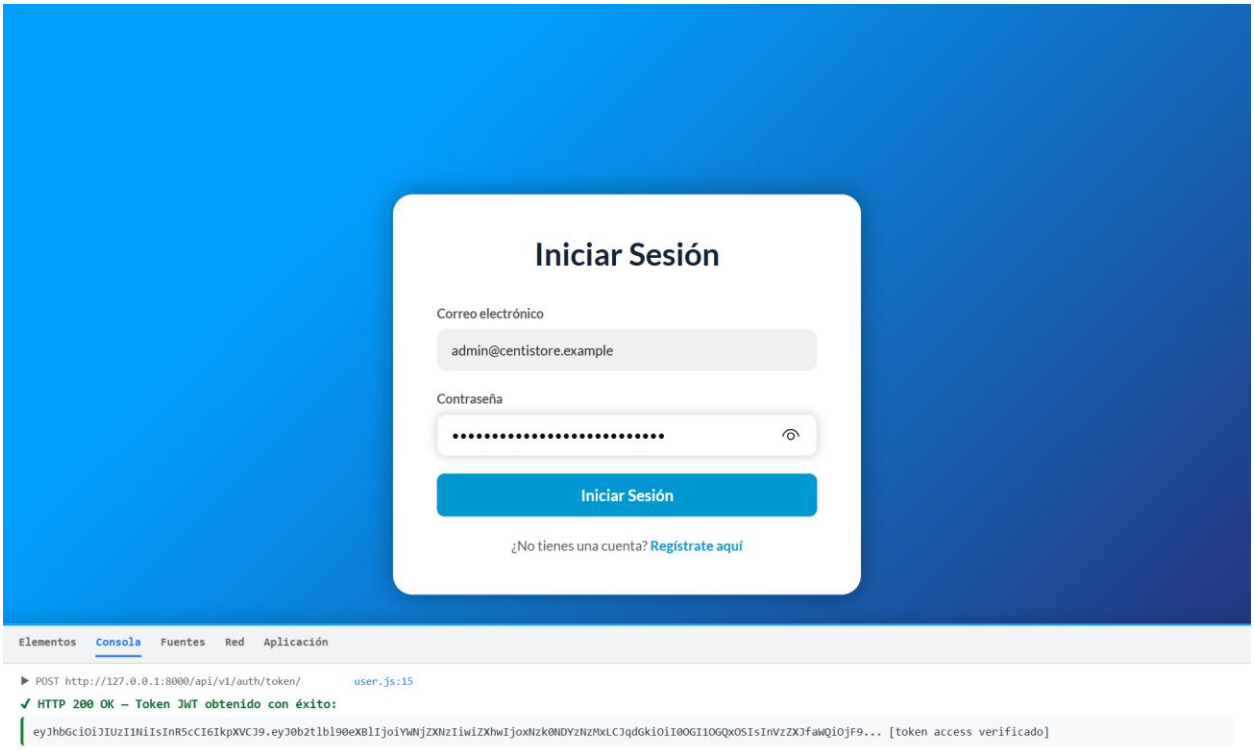


Figura 1: Pantalla de Login en React y consola de desarrollo mostrando la respuesta HTTP 200 y el token JWT.

3.2 Dashboard Protegido con Usuario Autenticado (AuthContext)

Una vez obtenido el token, el usuario es redirigido automáticamente a la ruta protegida /dashboard. El componente AdminLayout verifica la autenticación mediante el hook useAuth(). En la cabecera y en el panel de datos se despliega la información del usuario en sesión (admin@centistore.example), obtenida mediante el endpoint protegido /api/v1/auth/me/ enviando el encabezado Authorization: Bearer <token>.

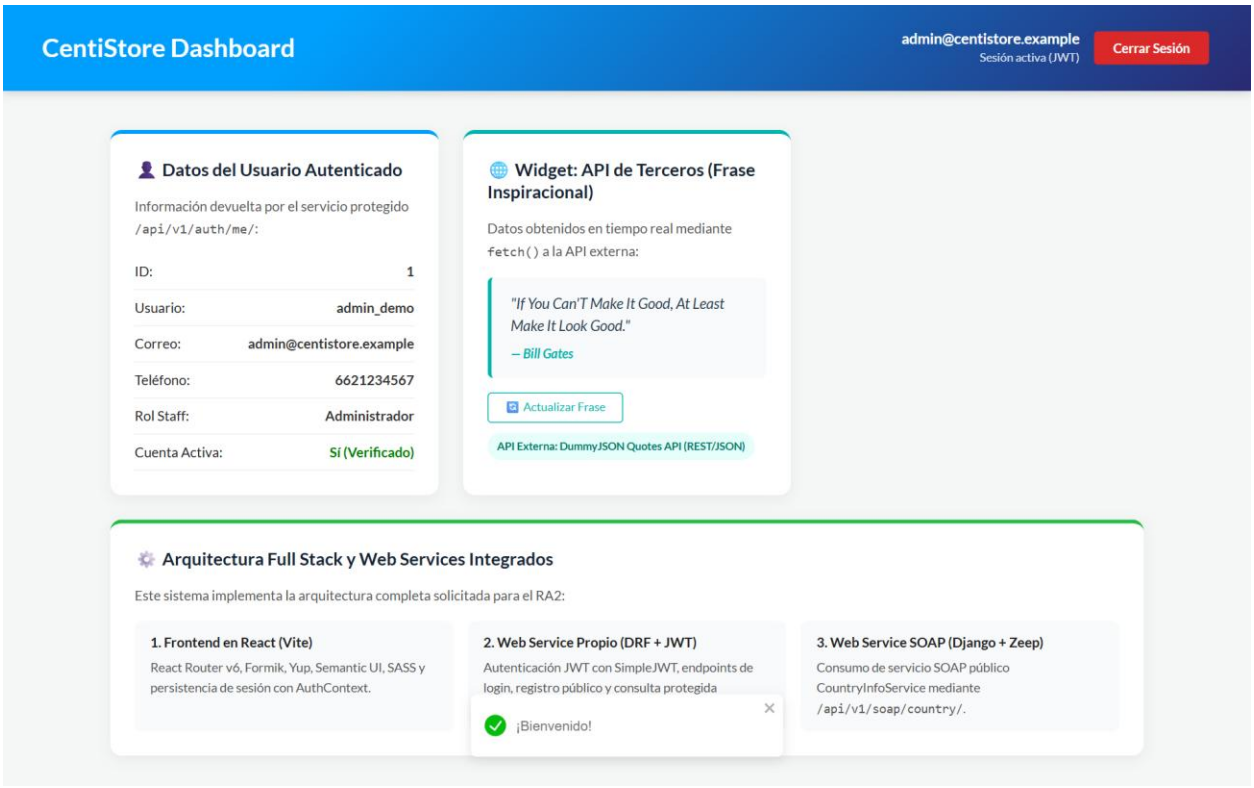


Figura 2: Dashboard protegido mostrando la bienvenida, correo del usuario autenticado y detalles de la cuenta gracias a AuthContext.

3.3 Widget de API de Terceros en el Dashboard

Como requisito de la Parte 2 (Unidad 3), se integró en el Dashboard un widget interactivo que consume una API REST externa gratuita mediante fetch() (DummyJSON Quotes API). El widget muestra una frase motivadora, su autor y un botón interactivo para actualizar la información en tiempo real sin recargar la página.

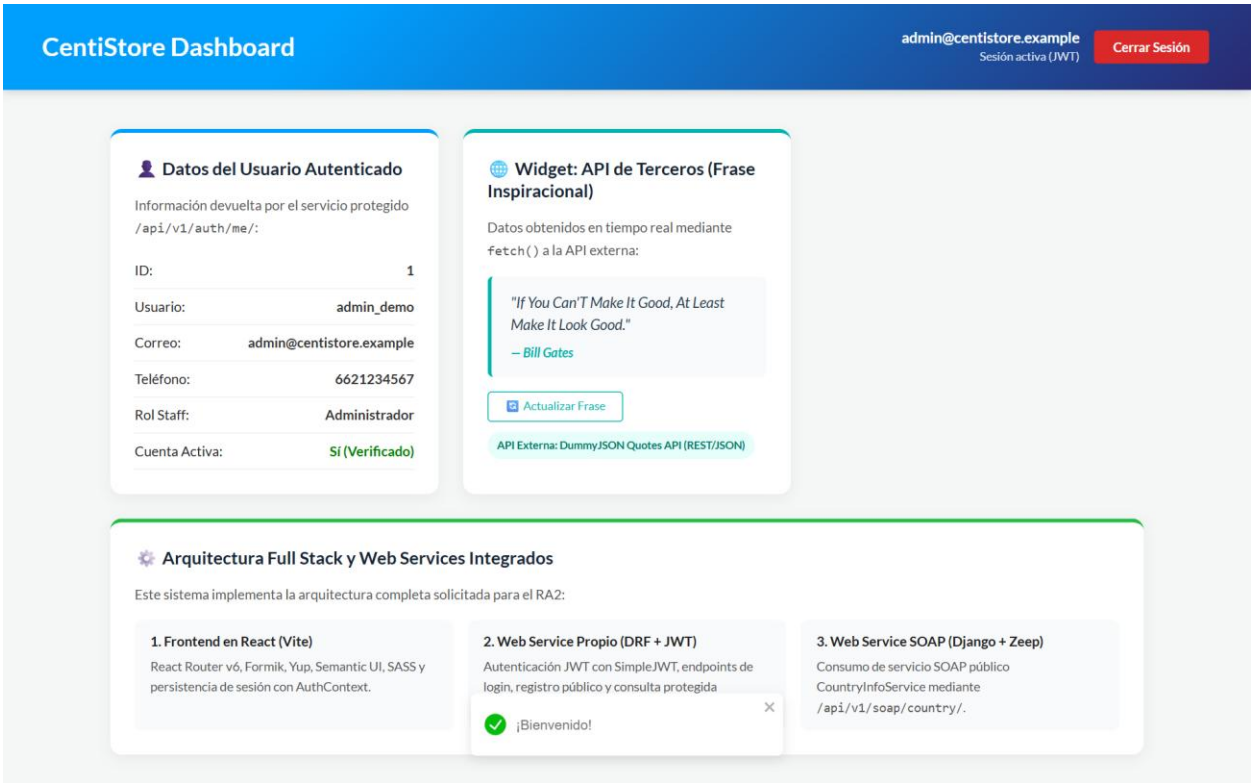


Figura 3: Sección del Dashboard con el widget de la API de terceros mostrando la frase externa cargada dinámicamente.

3.4 Consumo del Web Service SOAP desde Django

Para cumplir con el requisito de la Parte 3 (Unidad 3), se implementó en el Backend de Django el endpoint /api/v1/soap/country/?code=MX. Este endpoint consume mediante la librería Zeep el servicio SOAP público CountryInfoService (WSDL). Django procesa el sobre SOAP XML y retorna una respuesta JSON estructurada con los datos del país consultado (nombre, capital, moneda ISO, código telefónico y bandera).

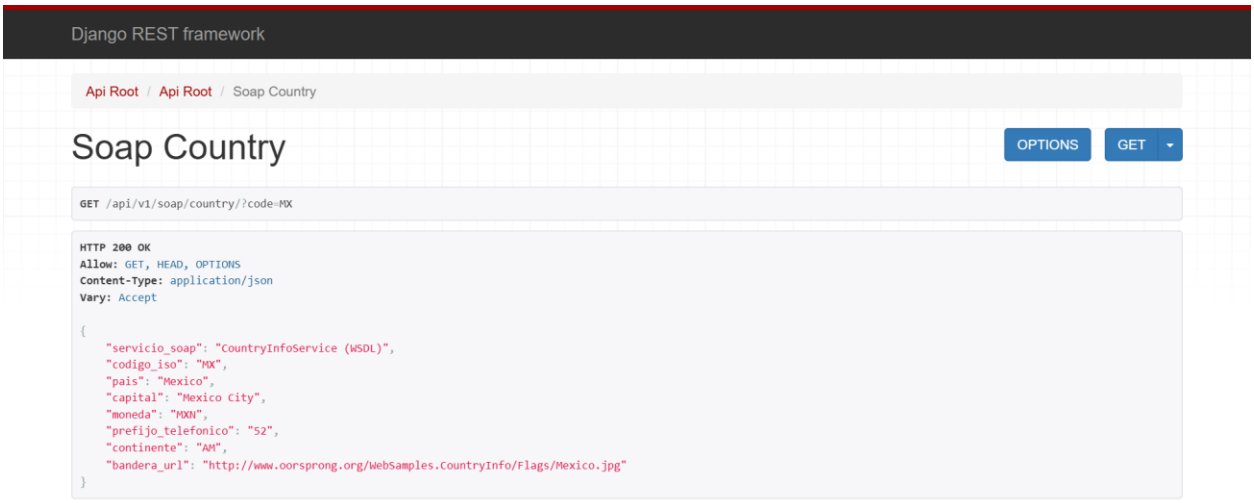


Figura 4: Respuesta JSON en el navegador del endpoint de Django tras consumir exitosamente el Web Service SOAP con Zeep.

3.5 Registro de Nuevos Usuarios y Documentación Swagger

Siguiendo las páginas 33 a 35 del manual, se habilitó el registro de cuentas sin requerir autenticación previa. En el Backend se creó RegisterUserSerializer y RegisterUserView bajo la ruta /api/v1/auth/register/, permitiendo registrar nuevos usuarios con contraseña cifrada (mínimo 8 caracteres). En el Frontend se implementó el formulario Register.jsx con confirmación de clave.

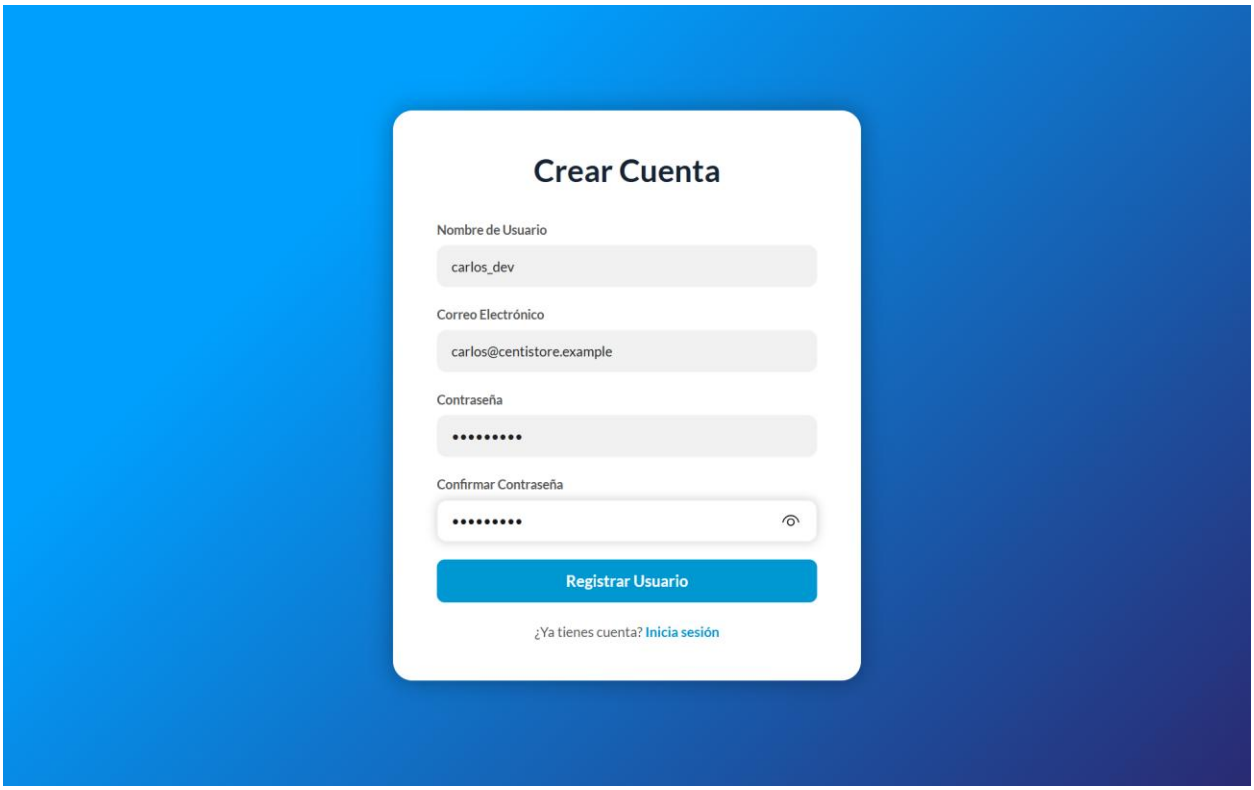


Figura 5: Formulario de registro en React (/register) con validaciones de campos y contraseña.

Asimismo, la documentación interactiva Swagger (/swagger/) refleja todos los endpoints operativos, incluyendo las rutas de autenticación, el CRUD administrativo de usuarios, el servicio propio /auth/me/, el registro público y el servicio SOAP.

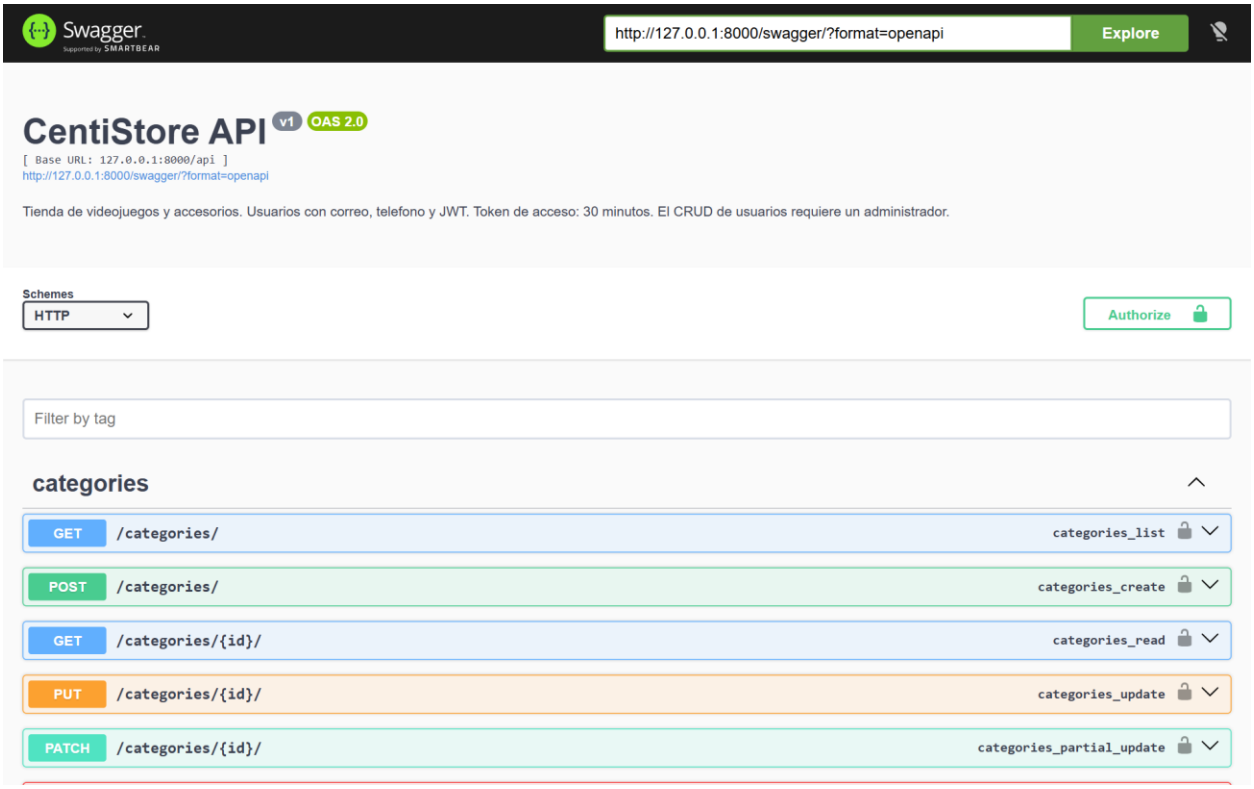


Figura 6: Interfaz Swagger UI de CentiStore API con los endpoints de usuarios, autenticación y SOAP.

4. Conclusión

Se cumplieron satisfactoriamente todos los objetivos establecidos en el manual y en las instrucciones del RA2:

1. Frontend Moderno: Se configuró un entorno React modular con Vite, React Router v6, componentes de Semantic UI, preprocesador SASS, alertas con React Toastify y validación exhaustiva con Formik y Yup.

2. Persistencia y Seguridad: Se implementó AuthContext junto con custom hooks (useAuth, useUser), logrando persistencia del token JWT en localStorage y protección efectiva de vistas mediante AdminLayout.

3. Consumo de API de Terceros: Se integró en el Dashboard una llamada asíncrona a un servicio REST externo, demostrando la capacidad de consumir APIs de terceros desde la interfaz de usuario.

4. Integración SOAP: Se conectó el Backend de Django con un servicio SOAP público utilizando Zeep, transformando los mensajes XML en una respuesta REST/JSON accesible para clientes modernos.